Software Architecture and Design Specification

Project: Smart Parking Management System (SPMS)

Version: 1.1

Authors: Team 06 – Raghav G Uddihal (PES1UG24AM215), Prithvish Adhikari (PES1UG24AM206), Rahul Prasanna (PES1UG24AM217), Rufus Thomas (PES1UG24AM232)

Date: 08-11-2026

Status: Final – as built after Sprint 2

Revision History

Version | Date | Author | Change Summary
1.0 | 07-10-2026 | Team 06 | Initial architecture & design: component and class diagrams, 2 sequence diagrams, API design, STRIDE threat model
1.1 | 08-11-2026 | Team 06 | As-built update: section 3.6 as built, 4.6 resolved issues, appendix 5.4 deviations

Approvals

Role | Name | Signature/Date
Course Coordinator | coordinator@pes.edu |

1. Introduction

1.1 Purpose
This document specifies the architecture and design of the Smart Parking Management System (SPMS).

1.2 Scope
Covers SPMS services: user registration/login, lot & slot management, real-time availability, slot reservation, automatic slot allocation, vehicle entry/exit, fee calculation & payments, reports and notifications.

1.3 Audience
Developers, QA engineers, security reviewers, instructors/evaluators, and maintenance teams.

1.4 Definitions
SPMS, Lot, Slot, Session, ADR, JWT, RBAC, ORM, STRIDE, TLS, WSS, UPI.

2. Document Overview

2.1 How to use this document
Provides architectural deliverables including UML component, class and sequence diagrams, ADRs, threat model, and API design.

2.2 Related Documents
SPMS SRS v1.0, SPMS STP v1.0, RTM (SRS section 8).

3. Architecture

3.1 Goals & Constraints

Goals: no double booking of a slot, live availability within 5 s, p95 API response ≤ 2 s, secure handling of personal and payment data, simple and testable modules. Constraints: Python (FastAPI) backend, React frontend, PostgreSQL; payment gateway in test mode only; no physical sensors/barriers; one-semester timeline for a team of four.

3.2 Stakeholders & Concerns

Drivers: finding/reserving a slot quickly, fair fees, payment security, privacy
Parking Attendants: fast and simple gate operations, allocation override
Administrator/Owner: accurate occupancy & revenue data, tariff control, auditability
Developers: modularity, testability, clear API contracts.
Instructor/Evaluators: traceability to requirements and sound design decisions.

3.3 Component (UML) Diagram

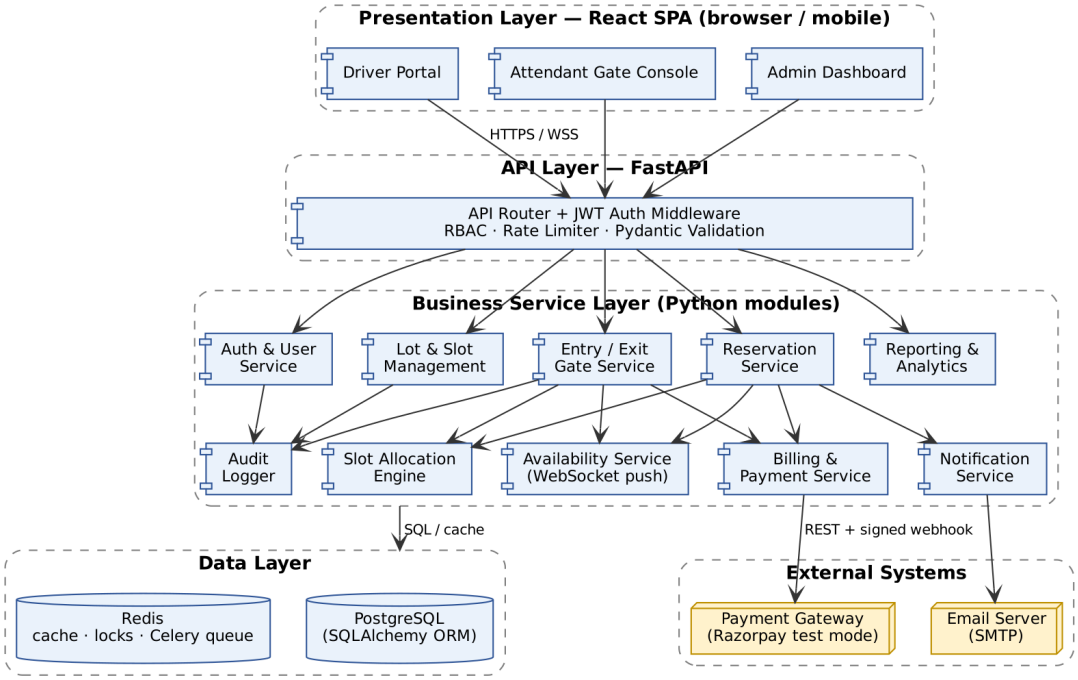


Figure 1: UML Component Diagram – layered architecture of SPMS

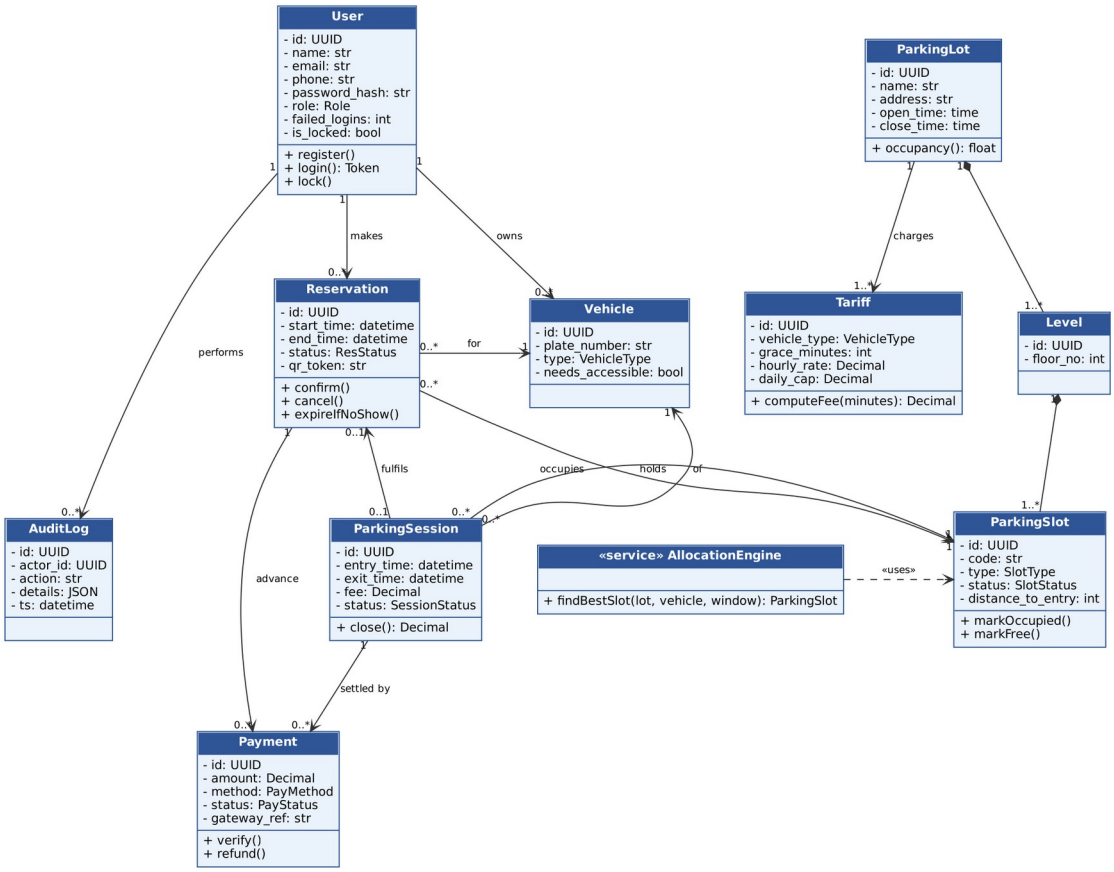


Figure 2: UML Class Diagram – SPMS domain model

3.4 Component Descriptions

- React UI (Driver Portal, Attendant Gate Console, Admin Dashboard): handles user input/output and live availability (WSS).
- API Router + Auth Middleware: routing, JWT validation, role-based access, rate limiting, input validation.
- Auth & User Service: registration, login, token refresh/revoke, account lockout, vehicle profiles.
- Lot & Slot Management: lots, levels, slots, tariffs, maintenance status.
- Availability Service: free-slot counters in Redis, pushes changes over WebSocket.
- Reservation Service: create/modify/cancel reservations, conflict checks, no-show expiry, QR tokens.
- Slot Allocation Engine: selects the best free compatible slot (type, accessibility, distance to entrance).
- Entry/Exit Gate Service: records entry/exit, validates QR, rejects duplicate entries, closes sessions.
- Billing & Payment Service: fee calculation, payment gateway orders/webhooks, cash

payments, refunds, receipts.
- Reporting & Analytics: occupancy, revenue, peak hours, driver history, CSV export.
- Notification Service: email confirmations and reminders (Celery worker + SMTP).
- Audit Logger: append-only log of privileged actions.
- Database: PostgreSQL (system of record) and Redis (cache, locks, pub/sub, task queue).

3.5 Chosen Architecture Pattern and Rationale

Layered architecture (modular monolith) chosen for clear separation of concerns: Presentation (React) → API (FastAPI routers) → Services (business modules) → Data access (SQLAlchemy) → PostgreSQL/Redis. Microservices rejected as overly complex for SPMS scale (multiple deployments and distributed transactions across reservation and payment). Key decisions: ADR-001 layered modular monolith; ADR-002 FastAPI for async I/O, WebSockets and auto OpenAPI docs; ADR-003 PostgreSQL for transactions, row locks and range exclusion constraints (no double booking); ADR-004 Redis pub/sub + WebSocket instead of polling; ADR-005 gateway-hosted checkout with signed webhooks to stay out of card-data scope.

3.6 Technology Stack & Data Stores

Python 3.11, FastAPI APIs, Pydantic, SQLAlchemy + Alembic, PostgreSQL 15 DB (exclusion constraint on slot + time range), Redis 7 (cache, locks, pub/sub) with Celery workers, React 18 frontend, JWT (python-jose) and bcrypt (passlib), TLS encryption via Nginx, Razorpay (test mode) payment gateway, SMTP email, Docker Compose deployment, GitHub Actions CI.

As built (v1.1): Python 3.12, FastAPI, Pydantic, SQLAlchemy with SQLite (PostgreSQL-ready through SPMS_DATABASE_URL), plain HTML/CSS/JavaScript pages served by FastAPI at /ui, PyJWT + bcrypt (cost 12), in-process background sweep (no-shows, reminders), in-memory rate limiter, WebSocket availability push every 2 s, mock payment gateway with HMAC-signed confirmation, email outbox with optional SMTP, GitHub Actions CI with a 70 % coverage gate. Double booking is prevented by a transaction plus an allocation lock instead of a PostgreSQL exclusion constraint. See appendix 5.4.

3.7 Risks & Mitigations

Risk: two users booking the last slot at the same time → Mitigation: row locks (SELECT … FOR UPDATE SKIP LOCKED) + DB exclusion constraint.
Risk: payment webhook delayed or lost → Mitigation: 10-minute hold with auto-expiry and a reconciliation job.
Risk: attendant enters wrong plate or misses an exit → Mitigation: plate validation, duplicate-entry check, audited admin correction.
Risk: Redis outage stops live updates → Mitigation: fall back to direct DB queries and 30 s client polling.
Risk: team schedule slip → Mitigation: build High-priority requirements first, weekly sprint reviews, CI regression tests.

3.8 Traceability to Requirements

F-001 (Driver registration), F-002 (Login & JWT issue), F-004 (Account lockout after 5 failures) → Auth & User Service

F-003 (Role-based access control) → API Router / Auth MW
F-010 (Manage lots / levels / slots), F-011 (Configure tariffs), F-012 (Slot maintenance status) → Lot & Slot Management
F-020 (Display free-slot counts), F-021 (Live status push ≤ 5 s) → Availability Service
F-030 (Reserve a slot), F-031 (Prevent double booking), F-032 (Modify / cancel with refund), F-033 (Auto-release no-shows) → Reservation Service
F-040 (Auto-allocate suitable slot), F-041 (Attendant allocation override) → Slot Allocation Engine
F-050 (Record vehicle entry), F-051 (Validate reservation QR), F-052 (Reject duplicate entry), F-053 (Record vehicle exit) → Entry/Exit Gate Service
F-060 (Fee calculation), F-061 (Online & cash payment), F-062 (Digital receipt), F-063 (Block unpaid exit / waiver) → Billing & Payment Service
F-070 (Occupancy & revenue dashboard), F-071 (CSV export), F-073 (Driver history & receipts) → Reporting & Analytics
F-072 (Email notifications) → Notification Service
SR-001 … SR-007 (security) → API Router / Auth Middleware, Audit Logger
NF-001 … NF-006 → overall architecture & deployment

3.9 Security Architecture

Threat Modeling (STRIDE): Spoofing → bcrypt passwords, account lockout, JWT, HMAC-signed payment webhooks; Tampering → fees computed only on server, input validation, parameterised ORM queries; Repudiation → append-only audit log; Info disclosure → TLS 1.2+, object-level authorisation, no card data stored; DoS → rate limiting, per-driver reservation limits; Elevation of Privilege → RBAC on every endpoint (deny by default).

4. Design

4.1 Design Overview

SPMS designed with layered modules (router → service → repository → model) for separation of concerns and easier maintenance. Slot states: FREE → RESERVED → OCCUPIED → FREE (MAINTENANCE as side state). Reservation states: PENDING_PAYMENT → CONFIRMED → CHECKED_IN → COMPLETED, or CANCELLED / NO_SHOW. Allocation picks the free compatible slot nearest to the entrance and locks it.

4.2 UML Sequence Diagrams

Two sequence diagrams covering 2 flows specific to SPMS: (1) Driver reserves a slot and pays online; (2) Vehicle entry and exit at the gate.

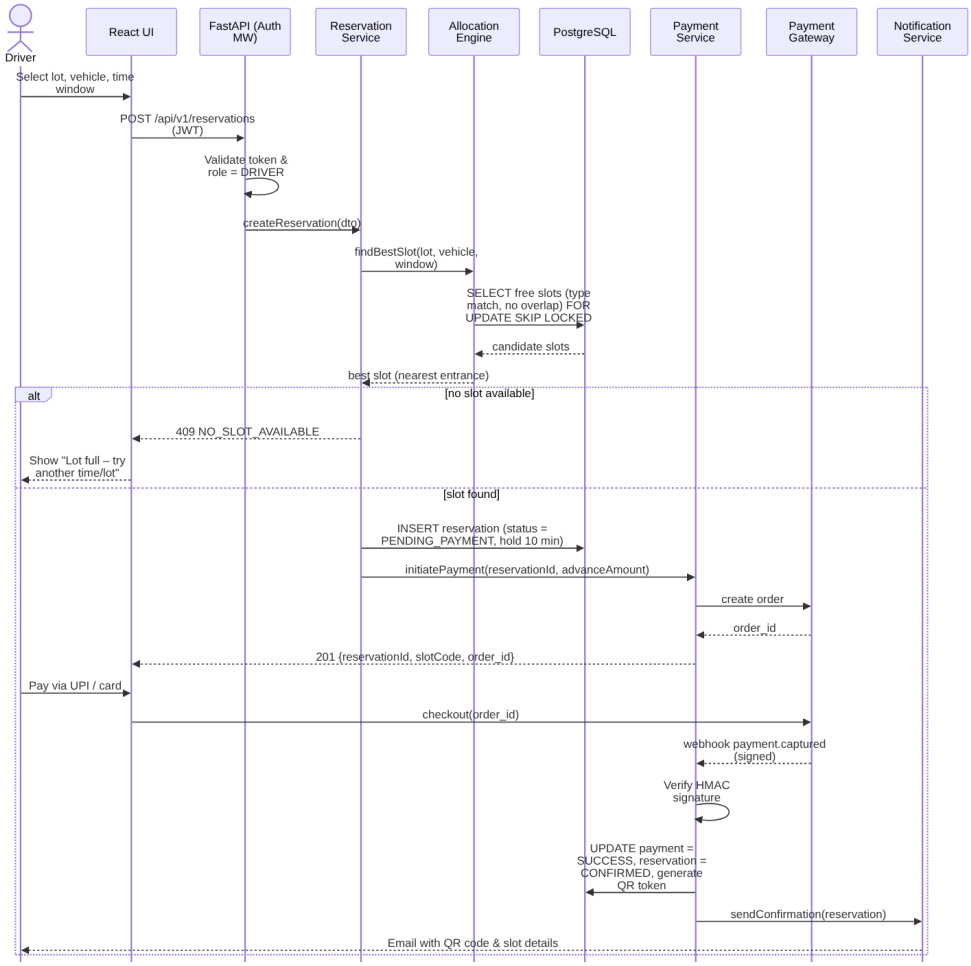


Figure 3: Sequence Diagram – Reserve slot with online payment

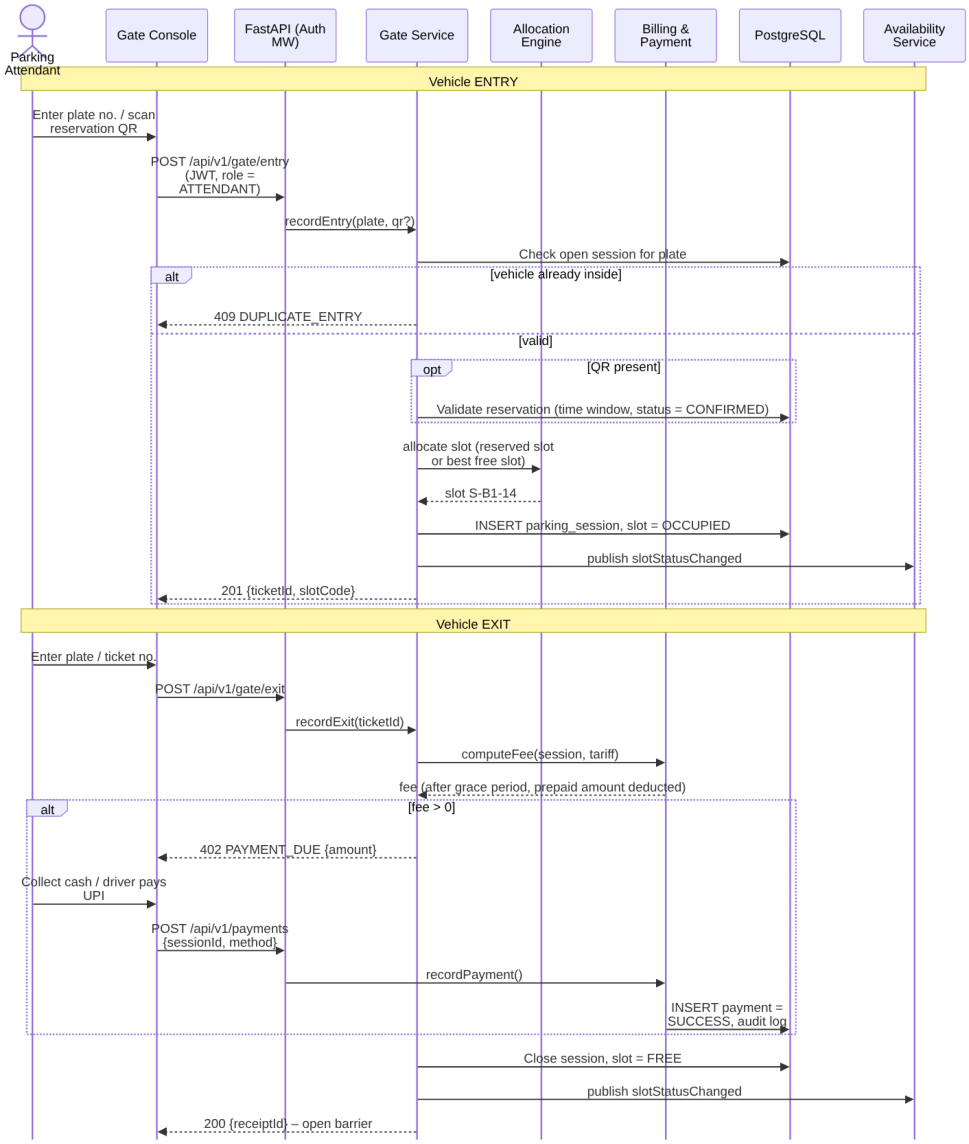


Figure 4: Sequence Diagram – Vehicle entry and exit with fee collection

4.3 API Design

Interface definition for at least 2 components (Reservation Service, Gate Service, Payment Service). Base path /api/v1, JSON over HTTPS, Bearer JWT.

Endpoint: /api/v1/reservations
Method: POST
Request: {lot_id, vehicle_id, start_time, end_time}
Response: {reservation_id, slot_code, status, advance_amount, payment_order_id}
Errors: 400 Invalid window, 401 Unauthorized, 409 No slot available, 422 Validation error.

Endpoint: /api/v1/gate/entry
Method: POST
Request: {lot_id, plate_number, vehicle_type, qr_token (optional)}
Response: {ticket_id, session_id, slot_code, entry_time}
Errors: 409 Duplicate entry, 409 Lot full, 422 QR invalid or expired.

Endpoint: /api/v1/gate/exit
Method: POST
Request: {ticket_id}
Response: {receipt_id, duration_min, fee, paid}
Errors: 402 Payment due {amount}, 404 Ticket not found.

Endpoint: /api/v1/payments/webhook
Method: POST (from payment gateway)
Request: gateway event + X-Razorpay-Signature header
Response: {status: processed}
Errors: 400 Invalid signature.

4.4 Error Handling, Logging & Monitoring

Standardized error messages: {error_code, message, request_id} with proper HTTP status; no stack traces to clients. No sensitive info in logs (passwords, tokens, payment payloads masked); structured JSON logs. Monitoring includes API latency (p95), error rate, occupancy per lot, failed payments and webhook delays via /health and /metrics.

4.5 UX Design

Mobile-first driver portal with colour-coded availability and a 3-step reservation wizard; tablet-friendly gate console with large Entry/Exit buttons and QR scan field; admin dashboard with KPI tiles and charts; accessible font sizes, WCAG 2.1 AA contrast and screen-reader labels.

4.6 Open Issues & Next Steps

Open issues: final refund percentages and sample tariffs; choice of Razorpay sandbox vs. mock gateway. Future enhancements: ANPR camera integration, IoT slot sensors, dynamic peak-hour pricing, EV-charger booking, native mobile app.

Resolved in v1.1: refund policy is 100 % at least 1 hour before the start, otherwise 50 %; the mock gateway was chosen over the Razorpay sandbox (same flow, signed confirmation).

5. Appendices

5.1 Glossary: SPMS, JWT, RBAC, ORM, STRIDE, TLS, WSS, UPI, ADR.
5.2 References: IEEE 42010, OWASP Top 10 / ASVS, NIST SP 800-160, FastAPI & PostgreSQL documentation, SPMS SRS v1.0.
5.3 Tools: PlantUML/Mermaid, Graphviz, draw.io, Swagger (FastAPI /docs).

5.4 Deviations from version 1.0 (as built)

Planned (v1.0)	As built (v1.1)	Impact
React 18 frontend	Plain HTML/CSS/JS pages at /ui	Same REST API; no build step
PostgreSQL 15 + exclusion constraint	SQLite via SQLAlchemy	Transaction + allocation lock; 50-parallel-request test
Redis 7 + Celery	In-process sweep, in-memory rate limit, WebSocket push	Single server per lot
Alembic migrations	create_all at start-up	Only new tables were added
python-jose, passlib	PyJWT, bcrypt	SR-002/003 unchanged
Razorpay sandbox	Mock gateway, HMAC-signed confirmation	SR-005 met, no card data
SMTP email	email_outbox table + optional SMTP	F-072 met
Nginx TLS, Docker Compose	uvicorn on localhost for the demo	SR-001 at deployment (reverse proxy)